

Билет №1

Тема: Разработка и интеграция модуля авторизации

Задание 1 (ПМ.01). Разработайте

класс `User` (поля: `id`, `login`, `password_hash`, `role`) и класс `AuthService` с методами `register()` и `login()`. Реализуйте проверку уникальности логина и хеширование пароля.

Задание 2 (ПМ.02). Создайте модуль `FileStorage` для сохранения коллекции пользователей в JSON-файл и загрузки из него. Интегрируйте его с `AuthService` так, чтобы данные автоматически сохранялись при регистрации.

Задание 3 (Отладка). Реализуйте класс `Logger` с методами `info()` и `error()`. Добавьте вызовы логирования в методы `register()` и `login()`. Сымитируйте ошибку ввода неверного пароля и продемонстрируйте запись ошибки в лог-файл.

Билет №2

Тема: Разработка модуля управления библиотечным фондом

Задание 1 (ПМ.01). Разработайте классы `Book` (название, автор, год, `isbn`) и `Library` с методами `add_book()`, `remove_book()`, `find_by_author()`. Используйте коллекции для хранения книг.

Задание 2 (ПМ.02). Реализуйте модуль для импорта книг из CSV-файла и экспорта в CSV. Обеспечьте обработку ошибок (неверный формат, отсутствие файла).

Задание 3 (Отладка). Добавьте в класс `Library` метод `validate_isbn()`, который проверяет корректность ISBN. Напишите юнит-тесты для этого метода (2-3 тестовых случая).

Билет №3

Тема: Разработка модуля расчёта заработной платы (ООП)

Задание 1 (ПМ.01). Разработайте иерархию классов: абстрактный класс `Employee` (поля: имя, ставка) и классы-наследники `HourlyEmployee` (почасовая оплата) и `SalaryEmployee` (оклад). Реализуйте метод `calculate_pay()` для каждого.

Задание 2 (ПМ.02). Создайте модуль `ReportGenerator`, который формирует текстовый отчёт по всем сотрудникам (имя, начисленная сумма). Интегрируйте его с системой расчёта зарплаты.

Задание 3 (Отладка). Реализуйте обработку исключений (отрицательная ставка, некорректные часы). Протестируйте работу с блоком `try-except`.

Билет №4

Тема: Разработка модуля логирования событий

Задание 1 (ПМ.01). Разработайте класс `Logger` с поддержкой трёх уровней логирования (INFO, WARNING, ERROR). Метод `log(level, message)` должен добавлять запись с временной меткой.

Задание 2 (ПМ.02). Реализуйте модуль `LogFileManager`, который автоматически создаёт новый файл лога каждый день и очищает логи старше 7 дней. Интегрируйте с классом `Logger`.

Задание 3 (Отладка). Напишите метод, который генерирует несколько исключений (деление на ноль, ошибка индекса). Покажите, как логгер фиксирует каждую ошибку с указанием типа исключения.

Билет №5

Тема: Разработка модуля работы с базой данных (SQLite)

Задание 1 (ПМ.01). Разработайте класс `Database` с методами `connect()`, `execute_query()`, `close()`. Создайте таблицу `products` (id, name, price, quantity).

Задание 2 (ПМ.02). Реализуйте модуль `ProductService` с методами `add_product()`, `get_all_products()`, `update_price()`. Используйте `Database` для выполнения SQL-запросов.

Задание 3 (Отладка). Добавьте обработку ошибок SQL (нарушение уникальности, неверный тип данных). Продемонстрируйте `rollback` при ошибке в транзакции.

Билет №6

Тема: Разработка модуля парсинга данных

Задание 1 (ПМ.01). Разработайте класс `LogParser`, который читает файл лога сервера. Метод `parse_ip()` должен извлекать все IP-адреса, метод `count_requests()` — подсчитывать количество запросов.

Задание 2 (ПМ.02). Реализуйте модуль `ReportBuilder`, который формирует отчёт о топ-5 самых частых IP-адресов в формате JSON. Интегрируйте его с `LogParser`.

Задание 3 (Отладка). Напишите тесты для проверки корректности извлечения IP-адресов из разных форматов строк лога (минимум 3 формата).

Билет №7

Тема: Разработка модуля для интернет-магазина (корзина)

Задание 1 (ПМ.01). Разработайте классы `Product` (`id`, `name`, `price`) и `Cart` с методами `add_item()`, `remove_item()`, `get_total()`, `get_item_count()`.

Задание 2 (ПМ.02). Реализуйте модуль `DiscountService`, который подключается к `Cart` и вычисляет скидку (например, 10% при сумме > 1000). Используйте паттерн "Декоратор".

Задание 3 (Отладка). Реализуйте проверку того, что нельзя добавить товар с отрицательной ценой. Напишите юнит-тест на это ограничение.

Билет №8

Тема: Разработка модуля экспорта данных

Задание 1 (ПМ.01). Разработайте класс `Report` с методами `add_row()` и `get_data()`. Реализуйте экспорт в CSV-формат через метод `to_csv()`.

Задание 2 (ПМ.02). Реализуйте модуль `PDFExporter`, который конвертирует отчёт из CSV в PDF. Используйте внешнюю библиотеку (`reportlab` или `fpdf`). Продемонстрируйте интеграцию.

Задание 3 (Отладка). Добавьте обработку ошибок при записи файла (недостаточно прав, диск заполнен). Покажите сообщение пользователю.

Билет №9

Тема: Разработка модуля шифрования данных

Задание 1 (ПМ.01). Разработайте класс `Encryptor` с методами `encrypt(text, key)` и `decrypt(encrypted, key)`. Используйте простой алгоритм (XOR или Caesar cipher) для демонстрации.

Задание 2 (ПМ.02). Реализуйте модуль `SecureStorage`, который сохраняет зашифрованные данные в файл и загружает их с расшифровкой. Интегрируйте с `Encryptor`.

Задание 3 (Отладка). Напишите тесты для проверки: `decrypt(encrypt(text)) == text` для разных входных данных.

Билет №10

Тема: Разработка модуля валидации данных

Задание 1 (ПМ.01). Разработайте класс `Validator` с методами: `is_valid_email()`, `is_valid_phone()`, `is_valid_date()`. Используйте регулярные выражения.

Задание 2 (ПМ.02). Реализуйте класс `FormValidator`, который принимает словарь с данными формы и схему валидации (список правил). Интегрируйте с `Validator` для проверки полей.

Задание 3 (Отладка). Напишите несколько тестов для метода `is_valid_email()`, включая граничные случаи (пустая строка, спецсимволы).

Билет №11

Тема: Разработка модуля очереди сообщений

Задание 1 (ПМ.01). Разработайте класс `MessageQueue` с методами `push(message)` и `pop()`. Реализуйте потокобезопасную версию с использованием блокировок.

Задание 2 (ПМ.02). Реализуйте модуль `QueueProcessor`, который обрабатывает сообщения из очереди (просто выводит их на экран). Интегрируйте с `MessageQueue`.

Задание 3 (Отладка). Сымитируйте ситуацию, когда очередь переполнена. Покажите, как система обрабатывает это состояние.

Билет №12

Тема: Разработка модуля кэширования

Задание 1 (ПМ.01). Разработайте класс `Cache` с методами `set(key, value, ttl)` и `get(key)`. Реализуйте автоматическое удаление устаревших записей.

Задание 2 (ПМ.02). Реализуйте модуль `CachedDataSource`, который обращается к источнику данных (файл/БД) и кэширует результаты. Продемонстрируйте работу с измерением времени выполнения.

Задание 3 (Отладка). Добавьте логирование событий попадания (cache hit) и промаха (cache miss).

Билет №13

Тема: Разработка модуля для управления заказами

Задание 1 (ПМ.01). Разработайте классы `Order` (статусы: new, paid, shipped, delivered) и `OrderManager` с методами `create_order()`, `update_status()`, `get_orders_by_status()`.

Задание 2 (ПМ.02). Реализуйте модуль `NotificationService`, который отправляет уведомления при смене статуса заказа (эмуляция: вывод в консоль). Интегрируйте с `OrderManager`.

Задание 3 (Отладка). Реализуйте проверку того, что статус заказа не может быть изменён на неподдерживаемое значение. Напишите тест на это ограничение.

Билет №14

Тема: Разработка модуля поиска и фильтрации

Задание 1 (ПМ.01). Разработайте класс `SearchEngine` с методом `search(items, query, field)`, который выполняет поиск по полю объекта. Используйте линейный поиск или бинарный (при условии сортировки).

Задание 2 (ПМ.02). Реализуйте модуль `FilterEngine`, который позволяет фильтровать коллекцию объектов по нескольким критериям (цена от-до, категория). Интегрируйте с `SearchEngine`.

Задание 3 (Отладка). Напишите тесты для проверки фильтрации: "вернуть товары с ценой от 100 до 500".

Билет №15

Тема: Разработка модуля резервного копирования

Задание 1 (ПМ.01). Разработайте класс `BackupManager` с методом `create_backup(source_dir, dest_dir)`, который копирует все файлы из директории-источника в директорию-назначения с добавлением временной метки.

Задание 2 (ПМ.02). Реализуйте модуль `RestoreManager`, который восстанавливает данные из выбранного бэкапа. Интегрируйте с `BackupManager`.

Задание 3 (Отладка). Добавьте обработку ошибок при копировании: что делать, если целевая директория не существует или нет прав на запись.

Билет №16

Тема: Разработка модуля для работы с API

Задание 1 (ПМ.01). Разработайте класс `APIClient` с методом `get(url)`, который выполняет HTTP GET-запрос (эмуляция или реальный запрос к тестовому API). Реализуйте обработку ответа (парсинг JSON).

Задание 2 (ПМ.02). Реализуйте модуль `WeatherService`, который обращается к публичному API погоды (можно эмулировать моком) и возвращает температуру. Интегрируйте с `APIClient`.

Задание 3 (Отладка). Добавьте обработку ошибок сети (таймаут, сервер недоступен). Покажите сообщение пользователю.

Билет №17

Тема: Разработка модуля статистики и аналитики

Задание 1 (ПМ.01). Разработайте класс `Statistics` с методами: `mean()`, `median()`, `mode()` для списка чисел. Реализуйте их самостоятельно, без использования готовых статистических библиотек.

Задание 2 (ПМ.02). Реализуйте модуль `DataAnalyzer`, который загружает данные из CSV-файла и вычисляет основные статистики по столбцам. Интегрируйте с `Statistics`.

Задание 3 (Отладка). Напишите тесты для метода `median()`: для чётного и нечётного количества элементов, для пустого списка.

Билет №18

Тема: Разработка модуля уведомлений

Задание 1 (ПМ.01). Разработайте классы для паттерна "Наблюдатель": интерфейс `Observer`, класс `Subject`. Реализуйте методы `attach()`, `detach()`, `notify()`.

Задание 2 (ПМ.02). Создайте конкретных наблюдателей: `EmailObserver` (выводит уведомление в консоль) и `LogObserver` (записывает уведомление в файл). Интегрируйте с `Subject`.

Задание 3 (Отладка). Реализуйте демонстрацию: событие "новый заказ" рассылается всем наблюдателям. Покажите результат.

Билет №19

Тема: Разработка модуля импорта/экспорта (XML/JSON)

Задание 1 (ПМ.01). Разработайте класс `ProductCatalog` с методами `add_product()`, `get_all_products()`, `find_by_id()`.

Задание 2 (ПМ.02). Реализуйте модули `JSONImporter` и `XMLImporter`, которые загружают данные из файлов разных форматов и создают объекты `Product`. Интегрируйте с `ProductCatalog`.

Задание 3 (Отладка). Добавьте валидацию импортируемых данных (проверка обязательных полей). Обработайте случай, когда файл имеет неверный формат.